



پردیس دانشکده‌های فنی

دانشگاه تهران



رباتیک پیشرفته - نیمسال دوم ۱۴۰۳-۴

فاز چهارم پروژه

نگارش:

علی خیری 810603071

معین رضایی 810602081

محمد رضا توکلی 159403003

استاد درس:

دکتر نصیری - دکتر مرادی

تابستان ۱۴۰۴

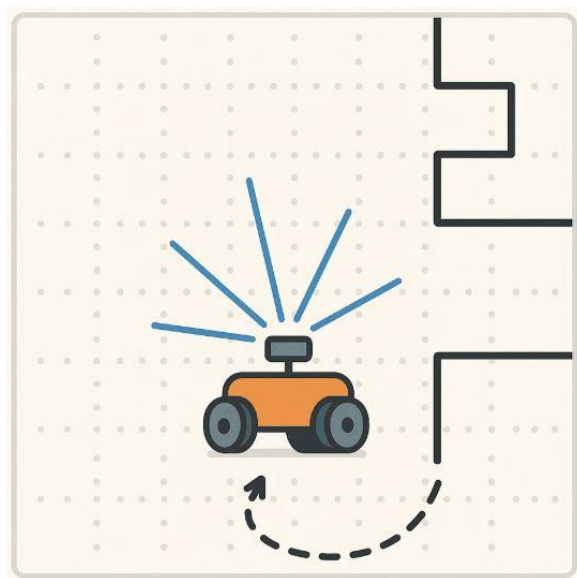
فهرست

۱ مقدمه

۱-۱ نقشه برداری و مکان یابی همزمان (SLAM)

نقشه برداری و مکان یابی همزمان یا SLAM یکی از موضوعات اساسی در رباتیک و سیستم های خودکار است. در این فرآیند ربات باید به طور همزمان موقعیت و جهت گیری خود را در محیطی ناشناخته تخمین بزند (مکان یابی) و در کنار آن نقشه ای از محیط اطراف خود بسازد یا به روزرسانی کند (نقشه برداری). این کار معمولاً با استفاده از داده های حسگرهایی مانند لیدار، دوربین استریو، سونار یا حسگرهای اینرسی انجام می شود. اهمیت SLAM زمانی بیشتر نمایان می شود که ربات هیچ اطلاعات اولیه ای از محیط یا موقعیت خود در اختیار نداشته باشد و ناچار باشد تنها بر اساس داده های حسگرها عمل کند. این فناوری کاربرد گسترده ای در ربات های خانگی، پهپادها، خودروهای خودران در محیط های ناشناخته دارد.

به طور کلی، الگوریتم های SLAM را می توان در چند دسته اصلی جای داد؛ از جمله روش های مبتنی بر فیلترهای احتمالاتی مانند EKF-SLAM یا FastSLAM و یا روش های مبتنی بر نقشه برداری ذره ای مانند Gmapping یا روش های متکی بر حسگر لیدار (LiDAR-SLAM) و روش های گراف محور (GraphSLAM) در ادامه، هر یک از این الگوریتم ها معرفی شده و ویژگی ها، مزایا و محدودیت های آن ها مورد بررسی قرار می گیرد. این مرور می تواند دید جامعی از سیر تحول الگوریتم های SLAM ارائه دهد و زمینه ای برای انتخاب مناسب ترین رویکرد در کاربردهای مختلف فراهم سازد.



۲-۱ انواع SLAM

پس از معرفی کلی مفهوم SLAM و بیان اهمیت آن در رباتیک و سیستم‌های خودکار، لازم است به بررسی رویکردهای مختلفی که تاکنون برای حل این مسئله پیشنهاد شده‌اند پرداخته شود. در طول دو دهه اخیر، پژوهش‌های گسترده‌ای در زمینه الگوریتم‌های SLAM صورت گرفته و روش‌های متنوعی با تکیه بر مدل‌های احتمالاتی و تکنیک‌های بهینه‌سازی توسعه یافته‌اند. هر یک از این روش‌ها با توجه به نوع داده‌های حسگری مورد استفاده مانند لیدار، دوربین یا IMU و همچنین شرایط محیطی (محیط‌های داخلی، خارجی، پویا یا ایستا) نقاط قوت و محدودیت‌های خاص خود را دارند.

❖ LiDAR-SLAM

به صورت کلی از حسگرهای لیدار برای اسکن محیط و ایجاد نقشه‌های دوبعدی یا سه‌بعدی با دقت بالا استفاده می‌کند و معمولاً با داده‌های IMU ترکیب می‌شود تا تخمین جهت‌گیری ربات بهبود یابد. این روش به دلیل توانایی کار در شرایط نوری مختلف (حتی تاریکی کامل) و تولید نقشه‌های بسیار دقیق، برای خودروهای خودران، پهپادها و ربات‌های اکتشافی در محیط‌های پیچیده مناسب است. با این حال، هزینه بالای حسگرهای لیدار، مصرف انرژی زیاد و حجم بالای داده‌ها از چالش‌های اصلی این روش به شمار می‌روند.

❖ GraphSLAM

این روش مبتنی بر لیدار می‌باشد. به این صورت که داده‌های لیدار و IMU را به صورت یک گراف مدل‌سازی می‌کند که در آن گره‌ها موقعیت‌های ربات و نقاط عطف محیط هستند. این روش با بهینه‌سازی کل مسیر و نقشه، خطاهای تجمعی را کاهش می‌دهد و در محیط‌های بزرگ و پیچیده بسیار دقیق عمل می‌کند. GraphSLAM به‌ویژه در کاربردهای صنعتی و نقشه‌برداری سه‌بعدی کارایی بالایی دارد. با این حال، پیچیدگی محاسباتی زیاد و نیاز به سخت‌افزار قدرتمند از محدودیت‌های آن محسوب می‌شود.

❖ EKF-SLAM (LiDAR and IMU-based)

از فیلتر کالمن توسعه یافته برای ترکیب داده‌های لیدار و IMU به منظور تخمین موقعیت ربات و ساخت نقشه استفاده می‌کند. این روش برای محیط‌های کوچک تا متوسط مناسب است و به دلیل سادگی نسبی و نیاز کمتر به منابع محاسباتی، در ربات‌های اولیه یا کم‌هزینه کاربرد دارد. با این وجود، در محیط‌های بزرگ یا با تعداد زیادی نقاط عطف، دقت آن کاهش یافته و خطاها به مرور زمان تجمع پیدا می‌کنند. به همین دلیل، در مقایسه با روش‌های مدرن‌تر مانند GraphSLAM یا FastSLAM کارایی کمتری دارد.

❖ FastSLAM (Particle Filter SLAM)

FastSLAM یک روش SLAM مبتنی بر فیلتر ذره‌ای است که از حسگرهای لیدار یا IMU برای تخمین همزمان موقعیت ربات و نقشه محیط استفاده می‌کند. این روش با نمونه‌برداری تصادفی (ذرات) عدم قطعیت‌های موقعیت‌یابی را مدیریت کرده و به‌ویژه در محیط‌های پویا و غیرخطی عملکرد مطلوبی دارد. FastSLAM به دلیل انعطاف‌پذیری در مدل‌سازی محیط‌های پیچیده برای ربات‌های متحرک مناسب است. با این حال، نیاز به تعداد زیادی ذره برای دستیابی به دقت بالا، بار محاسباتی را افزایش می‌دهد و در محیط‌های بزرگ یا دارای نویز زیاد، دقت آن کاهش می‌یابد.

❖ Gmapping

یک الگوریتم SLAM مبتنی بر فیلتر ذره‌ای است که به‌طور خاص برای داده‌های لیدار طراحی شده و اغلب با IMU ترکیب می‌شود تا دقت مکان‌یابی افزایش یابد. این روش با استفاده از اسکن‌های لیزری، نقشه‌های دوبعدی دقیقی از محیط تولید می‌کند و به دلیل پیاده‌سازی ساده و عملکرد قوی در محیط‌های داخلی مانند ساختمان‌ها، در ربات‌های خدماتی و صنعتی بسیار پرکاربرد است. همچنین Gmapping در سیستم‌هایی مانند ROS (Robot Operating System) به‌طور گسترده استفاده می‌شود. از معایب این روش می‌توان به افزایش خطای تجمعی در محیط‌های بزرگ یا متغیر و نیاز به تنظیم دقیق پارامترها اشاره کرد.

ذکر این نکته قابل توجه است که در این پروژه به دلیل اینکه در پروژه پیشین الگوریتم فیلتر ذرات پیاده سازی شد با الهام از روش Fast SLAM الگوریتم مورد مسیر یابی و مکان یابی همزمان را پیاده سازی کردیم.

۳-۱ چالش‌ها

با وجود اهمیت این فناوری، چالش‌های محاسباتی و فنی متعددی پیش روی آن قرار دارد. داده‌های حسگرها همواره با نویز و عدم قطعیت همراه هستند و الگوریتم مورد استفاده باید قادر به مدیریت این خطاها و ارائه تخمینی دقیق و پایدار از موقعیت ربات و ساختار محیط باشد. یکی از انواع چالش ما در پروژه نداشتن لیدار برای ارزیابی ۳۶۰ درجه محیط بوده است. متأسفانه در این پروژه باید از یک فاصله سنج لیزری استفاده می‌کردیم که تنها ۴۰ سانت حداکثر برد آن است. از طرفی به دلیل سنگین بودن این الگوریتم و حجم محاسبات بالای آن عملکرد این الگوریتم در سیستم‌های مختلف با سخت افزارهای ضعیف‌تر متفاوت است. از دیگر چالش‌هایی که در این پروژه با آن برخورد داشتیم خطای بالای ادومتری هنگام پایش محیط است این خطاها تجمعی ناشی از ادومتری

باعث شده است که حتی با داشتن رزولوشن بالا برای بدست آوردن نقشه کیفیت نقشه خروجی در مقایسه با نقشه حاصل از یک لایدار های ۳۶۰ درجه ضعیف تر باشد. که در بخش‌های آینده عملکرد این سیستم بررسی شده است.

۲ مروری بر الگوریتم فیلتر ذرات (Particle Filter)

فیلتر ذرات با مدل‌سازی موقعیت ربات به صورت مجموعه‌ای از «ذرات» کار می‌کند. هر ذره نماینده یک فرضیه درباره موقعیت و جهت‌گیری ربات (x, y, θ) است. این ذرات در ابتدا به صورت تصادفی در فضای آزاد نقشه پخش می‌شوند و با دریافت داده‌های حسگر (لیزر) و حرکت (اودومتری)، به‌روزرسانی می‌شوند تا به تدریج به موقعیت واقعی ربات همگرا شوند.



شکل ۱ - نقاط با موقعیت‌های مختلف سعی در تعیین موقعیت ربات دارند.

❖ Initialization

در ابتدای کار، تعدادی ذره به صورت تصادفی در نواحی آزاد نقشه پخش می‌شوند. هر ذره موقعیت و زاویه خاصی دارد و وزن اولیه همه آن‌ها برابر است. این ذرات در واقع فرضیه‌های اولیه از جایگاه احتمالی ربات محسوب می‌شوند.

❖ Motion Update

وقتی ربات حرکت می‌کند، داده‌های اودومتری باعث جابه‌جایی ذرات می‌شوند. این تغییر بر اساس مدل حرکتی ربات (سرعت خطی و زاویه‌ای) اعمال می‌شود و برای واقع‌گراتر شدن، نویز تصادفی نیز به آن افزوده می‌شود. این نویز باعث پخش بهتر ذرات و جلوگیری از همگرایی اشتباه می‌گردد.

❖ Sensor Update

در این مرحله، داده‌های حسگر مانند لیزر با نقشه مقایسه می‌شوند. هر ذره پیش‌بینی می‌کند که حسگر چه فاصله‌ای باید ببیند، و سپس این پیش‌بینی با داده واقعی مقایسه می‌شود. ذراتی که پیش‌بینی دقیق‌تری دارند وزن بیشتری می‌گیرند و معتبرتر می‌شوند.

❖ Resampling

ذرات با وزن پایین کنار گذاشته می‌شوند و ذرات معتبرتر تکثیر می‌گردند. برای اینکه تنوع حفظ شود و همه ذرات در یک نقطه جمع نشوند، تعدادی ذره با نویز تصادفی جابه‌جا می‌شوند. این کار باعث پایداری و جلوگیری از خطای همگرایی زودهنگام می‌شود.

❖ Pose estimation

موقعیت ربات از میانگین وزنی تمام ذرات به دست می‌آید. همچنین ذره‌ای که بیشترین وزن را دارد، به عنوان بهترین تخمین موقعیت فعلی ربات در نظر گرفته می‌شود.

❖ Convergence

وقتی تخمین موقعیت به واقعیت نزدیک شود و خطا کم باشد، الگوریتم وارد فاز همگرایی می‌شود. در این حالت، تعداد ذرات به مرور کاهش می‌یابد تا محاسبات سریع‌تر شود، و پراکندگی آن‌ها کمتر می‌شود تا ربات روی موقعیت درست متمرکز گردد. زمانی که خطا به حداقل برسد و ذرات بر روی موقعیت واقعی همگرا شوند، فرآیند محلی‌سازی کامل می‌شود. در این حالت، ذره با بیشترین وزن به عنوان موقعیت نهایی ربات اعلام و منتشر می‌شود.

۳ ساختار کلی SLAM

در این بخش، پارامترهای کلی الگوریتم SLAM مورد بررسی قرار گرفته‌اند. تمرکز بر دو جنبه اصلی است: اول، نحوه مدل‌سازی هر ذره و نقش آن در تولید نقشه و مکان‌یابی؛ دوم، فضای کلی که نقشه در آن برای ربات تولید می‌شود. این دسته‌بندی امکان تفکیک وظایف هر ذره و بهینه‌سازی فرایند ترکیب نقشه‌ها و همگرایی الگوریتم را فراهم می‌کند.

۱-۳ پارامترهای هر ذره

در این الگوریتم، برخلاف پیاده‌سازی استاندارد پارتیکل فیلتر که هر ذره تنها شامل موقعیت و جهت $(x,y,theta)$ بود، هر ذره به صورت مستقل وظیفه تولید یک نقشه مجزا را دارد. هر ذره با دریافت داده‌های سنسور لیزر، نقشه‌ای محلی ایجاد می‌کند و به‌روزرسانی می‌شود. پارامترهای هر ذره تعیین‌کننده تعداد ذرات اولیه و نهایی، میزان نویز حرکت و حسگر، آستانه‌های همگرایی و زمان‌بندی بررسی همگرایی هستند و به تعادل بین دقت و کارایی الگوریتم کمک می‌کنند.

پارامتر	مقدار	نقش پارامتر
INITIAL_PARTICLES	100	تعداد اولیه ذرات
MIN_PARTICLES	20	حداقل تعداد ذرات برای ادامه فیلتر
CONVERGED_PARTICLES	50	تعداد ذرات پس از همگرایی
FINAL_PARTICLES	20	تعداد ذرات نهایی پس از کاهش (downsampling)
LINEAR_STD	0.02 m	انحراف معیار نویز حرکت خطی.
ANGULAR_STD	0.05 rad	انحراف معیار نویز چرخش.
LASER_STD	0.05 m	انحراف معیار نویز حسگر لیزر.
MIN_TRANS_THRESH	0.01 m	حداقل جابجایی لازم برای آپدیت ذرات.
MIN_ROT_THRESH	0.02 rad	حداقل چرخش لازم برای آپدیت ذرات.

آستانه همگرایی اولیه؛ ذرات در این شعاع کاهش داده می‌شوند.	0.5 m	CONVERGENCE_THRESHOLD_1
آستانه همگرایی نهایی؛ ذرات به تعداد کمتر کاهش داده می‌شوند.	0.3 m	CONVERGENCE_THRESHOLD_2
حداقل تعداد خوانش‌های لیزر قبل از بررسی همگرایی	10	MIN_SAMPLES_FOR_CONVERGENCE
فاصله زمانی یا تعداد آپدیت بین بررسی همگرایی	20	CONVERGENCE_CHECK_INTERVAL
تعداد ذرات برتر برای ترکیب نقشه (map fusion)	5	TOP_PARTICLES_FOR_FUSION
فقط ذرات با وزن بیشتر از این آستانه در ترکیب نقشه استفاده می‌شوند	0.1	FUSION_WEIGHT_THRESHOLD
فاصله زمانی بین به‌روزرسانی‌های ترکیب نقشه.	0.3 s	FUSION_UPDATE_INTERVAL

۲-۳ پارامترهای هر نقشه

پارامترهای نقشه تعیین می‌کنند محیط ربات به چه صورت به سلول‌های گسسته تقسیم شود و اطلاعات مربوط به موانع و مسیرهای آزاد چگونه ذخیره و به‌روزرسانی شوند. این پارامترها شامل دقت سلول‌ها، ابعاد نقشه، موقعیت مبدا، مقادیر لاگ-اودز برای به‌روزرسانی وضعیت سلول‌ها، و همچنین تنظیمات صاف‌سازی و بررسی ثبات دیوارها هستند. تنظیمات ری‌کستینگ و اشتراک‌گذاری نقشه بین ذرات نزدیک نیز به بهبود کیفیت نقشه و کاهش نویز کمک می‌کنند. به این صورت که در فضای ۴۰۰ در ۴۰۰ با دقت ۰,۰۲ ما یک نقشه خالی ۶۴ متر مربعی داریم که در لحظه هر کدام از پارتیکل‌ها با داشتن اطلاعات سنسور‌ها سعی می‌کنند در این فضا نقشه مجزای خودشان را تولید کنند.

پارامتر	مقدار	نقش پارامتر
MAP_RESOLUTION	0.02 m/cell	اندازه هر سلول نقشه؛ دقت نقشه محیط
MAP_WIDTH	400 cells	عرض نقشه به تعداد سلول؛ محیط ۸ متر.
MAP_HEIGHT	400 cells	ارتفاع نقشه به تعداد سلول؛ محیط ۸ متر.
MAP_ORIGIN_X	None	مختصات X مبدا نقشه؛ می‌توان صفر یا دلخواه قرار داد.
MAP_ORIGIN_Y	None	مختصات Y مبدا نقشه
LOG_ODDS_OCC	1.7	افزایش مقدار لاگ-اودز وقتی مانع دیده شود.
LOG_ODDS_FREE	-0.2	کاهش مقدار لاگ-اودز وقتی مسیر خالی است.
LOG_ODDS_MIN	-6.0	حداقل مقدار لاگ-اودز.
LOG_ODDS_MAX	6.0	حداکثر مقدار لاگ-اودز.
MIN_HITS_FOR_OCC	2	تعداد دفعات مشاهده مانع برای ثبت سلول به عنوان اشغال شده.
SMOOTH_EVERY_UPDATES	50	هر چند آپدیت لیزر نقشه را صاف (smooth) کنیم.
SMOOTH_KERNEL	3x3	هسته برای صاف کردن نقشه و کاهش نویز.
WALL_CONSISTENCY_THRESHOLD	0.7	آستانه بررسی ثبات دیوارها در نقشه.
NEIGHBOR_OCCUPIED_THRESHOLD	3	حداقل سلول همسایه اشغال شده برای تایید مانع.
INITIAL_RAYCAST_STEP	0.06 m	گام اولیه ری‌کستینگ (Coarse) برای ذرات اولیه.
FINAL_RAYCAST_STEP	0.01 m	گام نهایی ری‌کستینگ (Fine) برای ذرات همگرا.
MAP_SHARING_DISTANCE	0.2 m	فاصله برای اشتراک نقشه بین ذرات نزدیک.
MAP_SHARING_ANGLE	0.2 rad	زاویه برای اشتراک نقشه بین ذرات نزدیک.

۴ فرآیند نقشه برداری (mapping)

ربات با استفاده از داده‌های سنسور لیزر و اطلاعات موقعیت (اودومتری) محیط را گام به گام “می‌بیند”. برای هر اندازه‌گیری، خط بین موقعیت ربات و نقطه‌ای که لیزر برخورد کرده رسم می‌شود (با الگوریتم Bresenham و سلول‌های روی این خط مشخص می‌شوند. سلول‌های بین ربات و مانع به عنوان مسیر آزاد و سلول انتهایی که مانع را نشان می‌دهد به عنوان سلول اشغال‌شده ثبت می‌شوند. این کار به ربات اجازه می‌دهد که به تدریج یک نقشه‌ی شبکه‌ای (occupancy grid) از محیط بسازد.

لاگ-اودز به ربات کمک می‌کند که با تجمع مشاهدات متعدد تصمیم بگیرد یک سلول پر است یا خالی. وقتی لیزر یک سلول را خالی می‌بیند، مقدار لاگ-اودز آن کاهش می‌یابد، و وقتی مانع دیده می‌شود، افزایش می‌یابد.

این مقادیر محدود شده‌اند min و max تا از افراط جلوگیری شود. با این روش، حتی اگر یک بار خطای سنسور رخ دهد، نقشه پایدار می‌ماند و فقط سلول‌هایی که بارها مشاهده شده‌اند به عنوان دیوار ثبت می‌شوند. در واقع یک احتمالی از پر یا خالی بودن و عددی بین ۰ (خالی بودن) تا ۱ (پر بودن سلول) می‌باشد.

$$p = 1 - \frac{1}{1 + e^t}$$

الگوریتم Bresenham برای رسم خط بین ربات و نقطه برخورد لیزر به این صورت عمل می‌کند که ابتدا موقعیت ربات روی شبکه سلولی مشخص می‌شود و نقطه هدف لیزر به عنوان مقصد تعیین می‌گردد. سپس الگوریتم گام‌به‌گام از ربات به سمت مقصد حرکت می‌کند و در هر گام تصمیم می‌گیرد که آیا لازم است سلول فرعی کنار مسیر هم تغییر کند تا خط صاف و دقیق باقی بماند. هر سلولی که روی مسیر قرار می‌گیرد به لیست مسیر اضافه می‌شود و سلول‌های قبل از نقطه برخورد به عنوان مسیر آزاد و سلول نقطه برخورد به عنوان مانع علامت‌گذاری می‌شوند. این روند ادامه پیدا می‌کند تا به سلول مقصد برسیم و در نهایت تمام سلول‌های مسیر برای به‌روزرسانی نقشه آماده می‌شوند. به این شکل الگوریتم، مسیر دقیق و پیوسته بین ربات و مانع را روی شبکه سلولی مشخص می‌کند و امکان آپدیت سریع و صحیح نقشه را فراهم می‌آورد.

نقشه‌های تولید شده توسط لیزر و اودومتری همیشه شامل نویز هستند. اسموپینگ با استفاده از هسته کوچک (3x3) مقادیر لاگ-اودز سلول‌ها را میانگین‌گیری می‌کند و نویز تصادفی را کاهش می‌دهد، بدون اینکه دیوارهای قوی خراب شوند. همچنین با بررسی تعداد سلول‌های همسایه، دیوارهای پایدار حفظ می‌شوند و سلول‌های تک یا پراکنده ضعیف می‌شوند. این کار باعث می‌شود نقشه نهایی هموارتر و واضح‌تر باشد و نمای بهتری از محیط ارائه کند.

۵ مدل حرکتی و مدل سنسوری ذرات

۵-۱ مدل حرکتی

در الگوریتم فیلتر ذرات برای SLAM، مدل حرکتی تعیین می‌کند که وقتی ربات حرکت می‌کند، ذرات چه تغییری در موقعیت خود خواهند داشت. هر ذره نماینده یک احتمال برای موقعیت واقعی ربات است و باید نشان دهد که با حرکت ربات، احتمال موقعیت ربات در چه محدوده‌ای توزیع شده است.

ایده اصلی مدل حرکتی این است که حرکت ربات همیشه دقیق نیست: چرخ‌ها ممکن است کمی لغزش داشته باشند، فرمان‌دهی موتورها دقیق نباشد و یا سطح زمین ناهموار باشد. بنابراین باید یک نویز کوچک به حرکت ذرات اضافه شود تا عدم قطعیت طبیعی حرکت واقعی ربات شبیه‌سازی شود.

مراحل الگوریتمی مدل حرکتی:

۱. دریافت داده‌های اودومتری: ابتدا سرعت خطی v و سرعت زاویه‌ای w ربات از سنسورهای اودومتری استخراج می‌شوند. این داده‌ها بیانگر جابجایی ربات از زمان آخرین به‌روزرسانی هستند.
۲. اضافه کردن نویز: به هر ذره کمی نویز خطی و زاویه‌ای اضافه می‌شود. نکته بسیار مهم درباره این بخش این است که این نویز بر اساس نویزهای بدست آمده در بخش عملی پروژه عددگذاری می‌شوند و باعث می‌شود ذرات کمی پراکنده شوند، حتی اگر ربات دقیقاً یک مسیر مستقیم داشته باشد. این موضوع باعث می‌شود که مدل حرکتی و در نهایت کل الگوریتم، به اودومتری وابسته می‌شود.
۳. محاسبه موقعیت جدید ذره:
 - اگر سرعت زاویه‌ای نزدیک به صفر باشد، ذره در مسیر مستقیم حرکت می‌کند و موقعیت جدید با ساده‌ترین رابطه‌ی حرکت خطی محاسبه می‌شود.
 - اگر ربات در حال چرخش باشد، ذره مسیر قوسی را دنبال می‌کند. شعاع قوس از نسبت سرعت خطی به زاویه‌ای محاسبه می‌شود و موقعیت ذره در طول قوس به‌روزرسانی می‌شود. زاویه ذره نیز با توجه به زاویه چرخش تغییر می‌کند.
۴. نرمال‌سازی زاویه: بعد از به‌روزرسانی، زاویه ذره به محدوده $[-\pi, \pi]$ برگردانده می‌شود تا زاویه‌های بزرگ یا کوچک غیرمعقول به درستی مدیریت شوند.
۵. نتیجه: پس از اجرای مدل حرکتی، ذرات به طور همزمان موقعیت جدید خود را به دست می‌آورند و یک ابر احتمال برای موقعیت ربات ایجاد می‌شود. ذراتی که نزدیک مسیر واقعی ربات هستند احتمال بیشتری برای بازنمایی موقعیت واقعی دارند، در حالی که ذرات دورتر احتمال کمتری دارند.

نکات مهم:

- مدل حرکتی باعث می‌شود که فیلتر ذرات تنها با داده‌های سنسوری کار نکند و توزیع اولیه ذرات براساس حرکت ربات تغییر کند.

- نويز در مدل حرکتی عدم قطعیت طبیعی حرکت ربات را شبیه‌سازی می‌کند و از همگرایی بیش از حد زودهنگام ذرات جلوگیری می‌کند.
- مسیر قوسی برای حرکت‌های زاویه‌ای باعث می‌شود ذرات دقیق‌تر با حرکت واقعی ربات مطابقت داشته باشند.

۵-۲ مدل سنسوری

مدل سنسوری وظیفه تعیین وزن هر ذره را بر عهده دارد. وزن ذره نشان‌دهنده‌ی میزان احتمال این است که آن ذره موقعیت واقعی ربات را نمایش دهد. این مدل با مقایسه داده واقعی سنسور لیزر و پیش‌بینی همان داده از روی نقشه داخلی ذره کار می‌کند.

مراحل الگوریتمی مدل سنسوری:

۱. پیش‌بینی فاصله تا مانع: برای هر ذره، با استفاده از نقشه داخلی خود، پیش‌بینی می‌شود که فاصله تا مانع در جهت لیزر چقدر است. این کار معمولاً با raycasting انجام می‌شود، یعنی از موقعیت ذره در جهت زاویه لیزر حرکت می‌کنیم تا اولین مانع را پیدا کنیم.
۲. مقایسه با داده واقعی: فاصله پیش‌بینی‌شده با فاصله واقعی اندازه‌گیری‌شده توسط لیزر مقایسه می‌شود. اگر فاصله‌ها مشابه باشند، به این معناست که نقشه داخلی ذره با واقعیت محیط همخوانی دارد.
۳. محاسبه وزن ذره:
 - اگر فاصله‌ها نزدیک باشند → وزن ذره افزایش می‌یابد.
 - اگر فاصله‌ها متفاوت باشند → وزن ذره کاهش می‌یابد.
 - اگر لیزر بیشینه برد داشته باشد و ذره هم فضای آزاد پیش‌بینی کرده باشد، باز هم وزن بالاست، زیرا پیش‌بینی با واقعیت مطابقت دارد.
۴. نرمال‌سازی وزن‌ها: پس از محاسبه وزن تمام ذرات، وزن‌ها نرمال‌سازی می‌شوند تا مجموع آن‌ها برابر با ۱ شود. این کار باعث می‌شود که ذرات بتوانند در مرحله resampling به نسبت وزنشان انتخاب شوند.
۵. نتیجه: ذراتی که موقعیت آن‌ها بیشترین تطابق با داده واقعی دارند، وزن بالایی دارند و احتمال بیشتری برای انتخاب در مرحله resampling دارند. ذراتی که تطابق ضعیف دارند حذف یا کمتر انتخاب می‌شوند.

نکات مهم:

- مدل سنسوری به ذرات اجازه می‌دهد که خود را با محیط تطبیق دهند و ذرات غیرممکن را حذف کند.

- ترکیب مدل حرکتی و مدل سنسوری باعث می‌شود که ذرات با گذشت زمان به موقعیت واقعی ربات همگرا شوند.
- این مدل امکان شناسایی موانع و به‌روزرسانی نقشه داخلی هر ذره را فراهم می‌کند.

۶ بروزرسانی وزن ذرات

در الگوریتم SLAM مبتنی بر فیلتر ذرات، هر ذره نماینده یک فرضیه از موقعیت و جهت ربات است و به طور همزمان یک نقشه محلی از محیط ایجاد می‌کند. به عبارت دیگر، هر ذره مانند یک ربات مستقل عمل می‌کند و با استفاده از داده‌های حسگر و مدل حرکت، محیط را شبیه‌سازی می‌کند. این ذرات به صورت پراکنده حول موقعیت اولیه ربات قرار می‌گیرند و به مرور زمان با حرکت ربات و دریافت داده‌های حسگر به‌روزرسانی می‌شوند. وزن‌دهی ذرات نقش مهمی در انتخاب ذرات معتبر برای نسل بعدی دارد و میزان اعتبار هر ذره نسبت به موقعیت واقعی ربات را نشان می‌دهد.

وزن هر ذره بر اساس اختلاف بین اندازه‌گیری واقعی حسگر و پیش‌بینی حسگر توسط ذره محاسبه می‌شود. فرض کنید Z_{actual} مقدار اندازه‌گیری شده توسط لیزر و $Z_{\text{predicted}}$ مقدار پیش‌بینی شده توسط ذره باشد. وزن ذره w با استفاده از مدل گاوسی به صورت زیر تعریف می‌شود:

$$w = \exp\left(-\frac{(Z_{\text{actual}} - Z_{\text{predicted}})^2}{2\sigma^2}\right)$$

در اینجا σ انحراف معیار مدل حسگر است که خطای احتمالی اندازه‌گیری‌ها را در نظر می‌گیرد. اگر اختلاف بین Z_{actual} و $Z_{\text{predicted}}$ کوچک باشد، وزن ذره نزدیک به ۱ خواهد بود و ذره به خوبی موقعیت واقعی را پیش‌بینی کرده است. اگر اختلاف زیاد باشد، وزن کاهش می‌یابد و احتمال بازنمونه‌گیری آن کمتر می‌شود.

پس از محاسبه وزن‌ها، آن‌ها نرمال‌سازی می‌شوند تا جمعشان برابر با ۱ شود:

$$w_i \leftarrow \frac{w_i}{\sum_{j=1}^N w_j}$$

این کار تضمین می‌کند که وزن‌ها به صورت احتمال تفسیر شوند و ذرات با وزن بالاتر بیشتر در نسل بعدی انتخاب شوند. ذرات با وزن بالا احتمال بیشتری برای باقی ماندن و تولید نسل بعدی دارند و ذرات ضعیف حذف می‌شوند. به طور شهودی، این فرمول ذراتی را که نقشه و موقعیت آن‌ها با داده واقعی حسگر هماهنگی بیشتری دارد، تقویت می‌کند و باعث بهبود دقت موقعیت و نقشه جهانی می‌شود.

برای نمایش بصری، رنگ ذرات بر اساس وزن آنها تعیین می‌شود؛ ذرات با وزن بالا به رنگ سبز و ذرات با وزن پایین به رنگ قرمز نمایش داده می‌شوند. این رنگ‌ها کمک می‌کنند تا به‌صورت شهودی مشاهده کنیم کدام ذرات به موقعیت واقعی ربات نزدیک‌تر هستند. در نهایت، نقشه جهانی از ترکیب نقشه ذرات با وزن‌هایشان به‌دست می‌آید و محیط کاوش‌شده، موانع و مناطق آزاد به صورت مستمر به‌روزرسانی می‌شوند، که الگوریتم را برای ناوبری و مسیریابی دقیق ربات آماده می‌کند.

۷ همگرایی و کاهش ذرات

در الگوریتم SLAM مبتنی بر فیلتر ذرات، هر ذره نماینده یک حدس از وضعیت ربات و نقشه محیط است. ذرات در ابتدا حول موقعیت شروع ربات پراکنده می‌شوند و هر ذره نقشه و وضعیت مختص خود را نگه می‌دارد. با حرکت ربات و دریافت داده‌های حسگر، ذرات به‌روزرسانی می‌شوند و وزن آنها براساس میزان تطابق پیش‌بینی‌های ذره با داده واقعی حسگر تعیین می‌شود. به صورت ریاضی، وزن ذره i را می‌توان به شکل زیر نوشت:

$$w_i = p(z_t | x_t^{[i]} m^{[i]})$$

پس از هر به‌روزرسانی حسگر، الگوریتم پراکندگی ذرات را محاسبه می‌کند. پراکندگی نشان می‌دهد که ذرات چقدر حول یک نقطه متمرکز شده‌اند. اگر ذرات هنوز پراکنده باشند، یعنی ربات هنوز موقعیت خود را به طور قطعی پیدا نکرده است. اما وقتی ذرات حول یک نقطه جمع شوند و پراکندگی کاهش یابد، نشان‌دهنده همگرایی ذرات است و ربات به یک حدس قابل اعتماد از موقعیت خود رسیده است. این مرحله باعث می‌شود سیستم بتواند تصمیم بگیرد که چه زمانی باید تعداد ذرات را کاهش دهد و محاسبات را سبک‌تر کند.

کاهش ذرات (Downsampling) به دو مرحله تقسیم می‌شود. در مرحله اول، وقتی ذرات پراکندگی کمی دارند، تعداد آنها از حالت اولیه به حدود ۵۰ ذره کاهش می‌یابد. در مرحله دوم، اگر همگرایی بیشتری مشاهده شود، تعداد ذرات به ۲۰ ذره می‌رسد. الگوریتم کاهش ذرات به صورت هوشمند عمل می‌کند: ذرات با وزن بالا همیشه حفظ می‌شوند تا بهترین حدس‌ها از بین نروند و باقی ذرات بر اساس تنوع مکانی انتخاب می‌شوند تا پوشش محیطی و احتمال خطای همگرایی محلی کاهش پیدا کند.

برای جلوگیری از انحصار ذرات در مناطق کوچک و حفظ دقت تخمین، از بازنمونه‌گیری سیستماتیک استفاده می‌شود. ذرات ضعیف حذف می‌شوند و ذرات قوی چندین بار کپی می‌شوند. این کار باعث تمرکز مجموعه ذرات روی موقعیت‌های محتمل و جلوگیری از پراکندگی غیرضروری می‌شود. بازنمونه‌گیری معمولاً زمانی انجام می‌شود که تعداد مؤثر ذرات یا

$$N_{\text{eff}} = \frac{1}{\sum_i w_i^2}$$

پایین تر از یک حد آستانه باشد، تا مطمئن شود هنوز تعداد کافی ذره معتبر برای نمایش موقعیت ربات موجود است.

در نهایت، این چرخه به صورت مداوم تکرار می شود :



شکل ۲ - حلقه الگوریتم SLAM

با گذشت زمان، ذرات به تدریج به سمت موقعیت واقعی ربات همگرا می شوند و نقشه محیط بهبود پیدا می کند. این فرآیند باعث می شود ربات بتواند با مجموعه محدودی از ذرات، موقعیت خود و نقشه محیط را به طور مؤثر و دقیق تخمین بزند.

۸ همجوشی و تولید نقشه نهایی

در همجوشی نقشه ها یا Map Fusion، ابتدا ذراتی که بهترین وزن را دارند انتخاب می شوند، زیرا وزن هر ذره نشان دهنده دقت موقعیت و کیفیت نقشه ای است که آن ذره ساخته است. وزن ها نرمال می شوند تا مجموعشان برابر یک شود و تاثیر هر ذره بر نقشه نهایی متناسب با احتمال صحیح بودنش لحاظ شود.

سپس برای هر سلول نقشه، اطلاعات تمامی ذرات در نظر گرفته می شود. به این صورت که سلول هایی که توسط چند ذره به عنوان اشغال شده شناسایی شده اند، اهمیت بیشتری پیدا می کنند و احتمال اشغال بودنشان بالاتر می رود. این کار باعث می شود نویز سنسورها و اشتباهات ناشی از ذرات کم کیفیت کاهش پیدا کند و یک تصویر یکپارچه تر از محیط شکل گیرد.

در نهایت، نقشه نهایی به صورت یک نقشه احتمالاتی تولید می‌شود و به ROS منتشر می‌شود تا سایر نودها بتوانند از آن استفاده کنند. هم‌زمان ذرات با رنگ و جهت مناسب برای نمایش بصری در RViz منتشر می‌شوند تا وضعیت ربات و پراکندگی ذرات به صورت قابل فهم و بصری قابل مشاهده باشد. این فرآیند باعث می‌شود که اطلاعات مکانی از ذرات متعدد در یک نقشه واحد و پایدار جمع‌آوری شود.

۹ ارزیابی الگوریتم:

برای ارزیابی الگوریتم ویدیوی SLAM قرار داده شده است. از این ارزیابی و بقیه تست‌های انجام شده نکات زیر قابل استخراج است:

۱- به دلیل بهینه‌سازی‌های انجام شده از جمله استفاده از Numpy، برداری‌سازی (Vectorization) محاسبات، تعداد نقاط کمتر نسبت به Particle Filter و بقیه موارد، الگوریتم علی‌رغم سنگین بودن، دارای سرعت قابل قبولی می‌باشد.

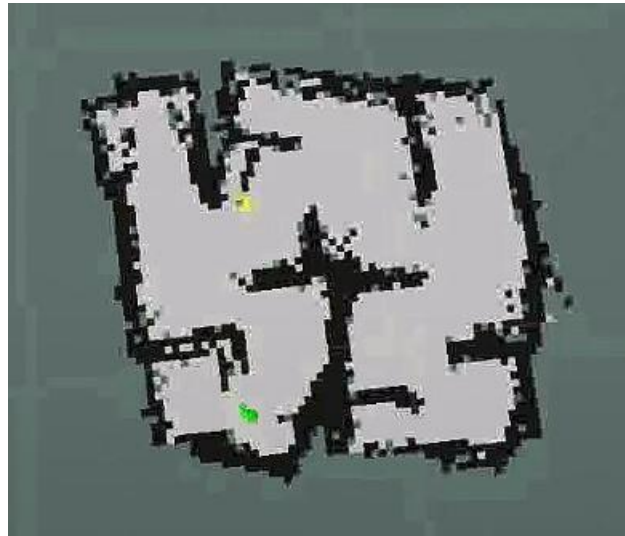
۲- سرعت خطی و زاویه‌ای ربات باید بسیار کم انتخاب شود تا فرآیند نقشه‌برداری دچار مشکل نشود.

۳- در اکثر مواقع، فرآیند مکان‌یابی زیر ۱ دقیقه انجام شده و تعداد ذرات از ۱۰۰ به ۵۰ و در نهایت ۲۰ کاهش یافته که موجب بهبود پردازش می‌شود.

۴- اکثراً نیاز به دور ۳۶۰ درجه می‌باشد.

۵- فرآیند نقشه‌برداری حدوداً ۳۰ دقیقه زمان برده و نیاز است تا کل نقشه حداقل دو بار اسکن شود.

۶- بدلیل دقت کم سنسور، خطای سنسور، خطای اودومتری، نقشه نهایی ایده‌آل نبوده و بخصوص در گوشه‌ها و نقاط پیوست دچار مشکل می‌شود:



شکل ۳ - نقشه نهایی تولیدشده

۱۰ پیشنهادات آینده

Loop Closure Detection زمانی رخ می‌دهد که ربات دوباره به منطقه‌ای که قبلاً دیده باز می‌گردد. شناسایی این حالت باعث می‌شود خطاهای تجمعی در موقعیت‌یابی کاهش یابند و تصحیح شوند. اگر این قابلیت به سیستم اضافه شود، نقشه نهایی دقیق‌تر و پیوسته‌تر خواهد شد و سلول‌هایی که به اشتباه آزاد یا اشغال شناسایی شده‌اند، اصلاح می‌شوند.

Pose Graph SLAM مفهومی است که موقعیت‌های ربات (pose) در طول زمان به صورت گراف مدل می‌شوند و ارتباط بین موقعیت‌ها به عنوان یال‌های گراف تعریف می‌شود، به ویژه زمانی که loop closure شناسایی می‌شود. به کمک این گراف، می‌توان بهینه‌سازی جهانی روی موقعیت‌ها انجام داد تا خطاهای تجمعی کاهش یافته و نقشه نهایی بسیار همگرا و دقیق‌تر شود. ترکیب این دو روش با سیستم فعلی باعث می‌شود نقشه نهایی از نظر ساختاری پایداری و قابل اعتمادتر باشد و ذرات بهتر بتوانند موقعیت واقعی ربات را نمایش دهند.

